Kosmocrates HYPHAE LPCM Extension

v0.4.2 – Controlled Fragmentation, Local-Percolative Collapse, and Agentic Corpus Cultivation

Companion extension to HYPHAE v0.4.1 and the Kosmocrates Workbench Master Specification

Author: Sebastian Klemm
30. May 2026

Document status. This document is an implementation-oriented extension. It integrates Local-Percolative Collapse Mechanics (LPCM) into HYPHAE v0.4.1 and adds the mechanisms discussed after v0.4.1: good fragmentation, monotone contractive reduction of spurious degrees of freedom, synthetic corpus cultivation, fixpoint surfing, and bounded host-centric structural condensation. It does not replace HYPHAE v0.4.1. It specifies a collapse/fragmentation layer that can be implemented under HYPHAE and used by Kosmocrates Workbench and Foundry.

Contents

Executive Summary

HYPHAE v0.4.1 introduced microtopological diagnosis and surgery planning below the persistent corpus cartography of v0.4. This extension adds the missing collapse mechanics that explain how fragmented external evidence, synthetic agent artifacts, local microtopology diagnostics, and candidate operations can be converted into stable, higher-order implementation candidates without pretending that local coherence is truth.

The extension is based on LPCM: Local-Percolative Collapse Mechanics. LPCM defines how a fragmented and underdetermined solution landscape is partitioned into bounded patches, triangulated into local topology, mapped into candidate directions, scored by normalized admissible support mass, passed through a strict local majority threshold above one half, linked through seam compatibility, and recursively coarse-grained into higher-order collapse candidates.

The governing phrase of this extension is:

> **Monotone contractive reduction of spurious degrees of freedom until structural condensation occurs.**

A spurious degree of freedom is not every freedom. It is a freedom that exists only because the current solution space is underdetermined, poorly triangulated, unsupported, unverified, semantically lossy, or not yet constrained by evidence, seams, gates, replay, or Foundry validation. HYPHAE v0.4.2 does not reduce valid design freedom indiscriminately. It reduces unsupported freedom until only structurally admissible candidate directions remain.

Core mechanism

```
Controlled Fragmentation
-> admissible fragment field
-> candidate directions per fragment
-> normalized support mass
-> local 51% dominance bits
-> seam-compatible percolation
-> coarse-grained collapse candidates
-> surgery-backed collapse plan
-> Workbench/Foundry materialization
-> Kosmocrates memory update
```

Why this extension matters

HYPHAE already knows how to search, extract, certify, remember, and diagnose. This extension defines how the system should fragment a large problem into locally collapsible evidence fields, how it should use external and synthetic artifacts as candidate support, how it should prevent local dominance from becoming false truth, and how it should lift many local collapses into a coherent HostTargetCollapsePlan.

The result is not a generic voting system. It is a controlled collapse layer for software evolution:

```
variation expands the phase space;
monotone filtering removes spurious degrees of freedom;
51%-dominant local patches emit condensation candidates;
```

```
seam percolation connects compatible local condensates;
coarse-graining forms higher-order structural plans;
Foundry and Kosmocrates decide what materializes.
```

Part I

Doctrine

1 Purpose and Scope

1.1 Purpose

This document specifies an extension layer that maps LPCM into HYPHAE/Kosmocrates. It defines how controlled fragmentation, local majority condensation, seam percolation, monotone contraction, synthetic corpus cultivation, and fixpoint surfing are represented as implementation concepts.

1.2 Scope

This extension covers:

- controlled fragmentation of a host problem into bounded local patches;
- admissible evidence construction from host code, external sources, prior memory, and synthetic agent artifacts;
- support-mass computation over local candidate directions;
- a local 51 percent dominance gate that creates candidates, not truth;
- seam compatibility and percolative paths;
- coarse-graining into meso-level and host-level collapse candidates;
- monotone filtering of spurious degrees of freedom;
- agentic corpus cultivation as controlled synthetic SourceCube generation;
- integration with Metatron microtopology diagnostics;
- Workbench/Foundry materialization constraints;
- data types, gates, algorithms, metrics, APIs, CLI commands, tests, and Definition of Done.

1.3 Non-goals

This extension is not:

- a replacement for HYPHAE v0.4.1;
- a replacement for Kosmocrates governance;
- a replacement for Foundry validation;
- a truth oracle;
- a global vote over repositories;
- a justification for raw code import;
- an autonomous patch executor;
- a model training procedure;
- a license-bypass or security-bypass mechanism.

1.4 Canonical statement

> **HYPHAE v0.4.2 treats fragmentation as a controlled expansion phase and LPCM as the contraction mechanism that turns admissible local support into seam-compatible higher-order collapse candidates.**

2 Terminology

2.1 Spurious degrees of freedom

> **Spurious degrees of freedom are apparent solution options that persist only because the landscape has not yet been sufficiently constrained by evidence, topology, support, seam compatibility, replay, gates, or validation.**

Examples:

- a candidate with no provenance;
- a candidate that exists only in opaque LLM text;
- a candidate that fits a fragment but breaks seams;
- a graph match caused by semantic loss;
- a source motif blocked by license or taint;
- a surgery option that cannot be validated by Foundry;
- an external pattern that looks common but fails host alignment.

2.2 Good fragmentation

Good fragmentation is not arbitrary splitting. It is the creation of independently inspectable, admissible, recombinable, evidence-carrying local patches.

> **Good fragmentation creates a maximum number of useful degrees of freedom while preserving the ability to filter, compare, validate, and recompose them.**

2.3 Structural condensation

Structural condensation is the point at which enough spurious degrees of freedom have been removed that the remaining candidate space becomes locally forced into a stable structural direction.

2.4 Fixpoint surfing

Fixpoint surfing is the repeated traversal of partial local collapses. The system does not require a global fixpoint at the beginning. Instead, it uses each local condensation to constrain adjacent regions until a percolative chain or higher-order collapse emerges.

2.5 Synthetic corpus cultivation

Synthetic corpus cultivation is the controlled generation of candidate artifacts by one or more agents for the purpose of structural comparison, filtering, and assimilation. These artifacts are treated as synthetic SourceCubes, not trusted outputs.

3 Relation to LPCM

3.1 LPCM core

LPCM defines a domain-open mechanism:

```
Evidence landscape
-> deterministic fragment partition
-> triangulated local patches
-> candidate directions
-> normalized support mass vectors
-> local 51% condensation bits
-> seam percolation graph
-> coarse-grained condensates
-> global collapse candidate or diagnostic hold
```

The 51 percent threshold is a local dominance criterion over admissible support mass. It is not a truth criterion and not materialization authority.

3.2 HYPHAE mapping

LPCM concept	**HYPHAE/Kosmocrates concept**
Solution landscape	Host-specific solution phase space / repository improvement search space
Evidence carrier	Host observations, SourceCubes, SyntheticSourceCubes, prior StructuralCrystals, Foundry results
Fragment partition	TopologyRegionSet / WorkbenchHDAG partition / FragmentField
Triangulated local patch	MetatronMicrograph / CodeHDAG window / TopologyRegion
Candidate direction	SurgeryOption / MotifCandidate / HostTargetDelta fragment
Support mass vector	Weighted evidence over candidate directions
Local 51 percent bit	LocalCondensationCandidate
Seam graph	compatibility graph between region candidates
Percolative path	connected compatible collapse path across host regions
Coarse-grained condensate	HostTargetCollapsePlan / CompositeSupportPlan

3.3 LPCM invariants adopted here

1. Evidence before support.
2. Topology before candidate.
3. Support before majority.
4. Local majority before percolation.
5. Seam before coarse-grain.
6. No local truth.
7. Replay identity.

Invariant LPCM-001: Local majority is not truth. A local 51 percent condensation bit **MUST NOT** be interpreted as truth, correctness, materialization authorization, or memory-promotion authority.

Invariant LPCM-002: Majority over admissible support only. A local majority calculation **MUST** be computed over normalized admissible support mass. Evidence without provenance, topology, gate status, or admissibility **MUST NOT** contribute to support mass.

Invariant LPCM-003: Seam before system-level collapse. Local collapse candidates **MUST** satisfy seam compatibility before they may be coarse-grained into host-level collapse plans.

Part II

Architecture

4 Layer Position

4.1 Stack position

```
External Corpus / Synthetic Agent Artifacts / Host Repository
        |
        v
HYPHAE Acquisition + SourceCube / SyntheticSourceCube Construction
        |
        v
CodeHDAG + Metatron Microtopology Projection
        |
        v
LPCM Fragmentation and Local Collapse Layer   <-- this extension
        |
        v
CubeSwarm / CubeMandorla / StructuralYield / StructuralCrystalCandidate
        |
        v
CollapsePlan + Workbench + Foundry
        |
        v
Kosmocrates Memory / CorpusCartography / NormGenes / Agenda
```

4.2 Responsibility split

Layer	Responsibility
HYPHAE Acquisition	Finds host-relevant source material and synthetic artifacts.
CodeHDAG	Represents structural code topology.
Metatron Microtopology	Projects bounded regions and diagnoses local shapes.
LPCM Extension	Converts fragmented support into local and percolative collapse candidates.
Workbench	Converts plans into bounded tasks and patches.
Foundry	Tests, checks, lints, and validates materialization.
Kosmocrates	Persists evidence, crystals, NormGenes, governance outcomes, and replay identity.

5 Fragment Field

5.1 Definition

A FragmentField is a deterministic decomposition of a host-relevant solution landscape into local fragments.

Type FragmentField.

```
pub struct FragmentField {
    pub id: Digest,
    pub host_id: Digest,
    pub goal_id: Digest,
    pub partition_policy: PartitionPolicy,
    pub fragments: Vec<FragmentPatch>,
    pub overlap_model: OverlapModel,
    pub evidence_scope: EvidenceScope,
    pub replay_descriptor: ReplayDescriptor,
}
```

Type FragmentPatch.

```
pub struct FragmentPatch {
    pub id: Digest,
    pub region_ref: TopologyRegionRef,
    pub boundary: BoundaryModel,
    pub local_observations: Vec<EvidenceRef>,
    pub triangulation: Option<LocalTriangulation>,
    pub candidate_directions: Vec<CandidateDirection>,
    pub status: FragmentStatus,
}
```

5.2 Fragment requirements

Requirement FRAG-001: Deterministic partition. Fragment partitioning **MUST** be deterministic under identical host state, goal, policies, operator versions, and seed.

Requirement FRAG-002: Traceable overlap. Fragments **MAY** overlap, but overlap **MUST** be explicit, traceable, and included in seam compatibility checks.

Requirement FRAG-003: Bounded local patch. Each fragment **MUST** declare its boundary, evidence scope, region reference, and topology construction policy.

6 Candidate Directions

6.1 Definition

A CandidateDirection is a locally admissible structural direction: completing, refining, contracting, splitting, inserting, isolating, testing, or rejecting a local patch.

Type CandidateDirection.

```
pub struct CandidateDirection {
    pub id: Digest,
    pub fragment_id: Digest,
    pub direction_kind: CandidateDirectionKind,
    pub generated_from: Vec<EvidenceRef>,
    pub required_surgery: Option<TopologicalSurgeryOptionId>,
    pub expected_effect: ExpectedStructuralEffect,
    pub validation_template: ValidationTemplate,
    pub risk_class: RiskClass,
}

pub enum CandidateDirectionKind {
    AddMissingEdge,
    RemoveSpuriousEdge,
    InsertMediatorRole,
    SplitOverloadedRole,
    MergeEquivalentRoles,
    AddTestFiber,
    AddSecurityBoundary,
    AddLayerBoundary,
    AddConstraint,
    MarkAntiPattern,
    EvidenceOnlyHold,
}
```

Requirement DIR-001: Candidate directions are hypotheses. Candidate directions are hypotheses about local structural completion. They **MUST NOT** directly generate patches or alter host state.

Requirement DIR-002: Direction evidence. Every CandidateDirection **MUST** cite at least one admissible evidence source, or it **MUST** be marked as speculative and excluded from majority support unless policy permits speculative analysis.

7 Support Mass

7.1 Support factors

For a candidate direction c_i inside a patch U, HYPHAE computes an unnormalized support mass:

$$\tilde{w}_i^U = A_i \cdot B_i \cdot S_i \cdot R_i \cdot P_i \cdot G_i \cdot H_i \cdot F_i \cdot M_i \tag{7.1}$$

where:

Factor	Meaning
A_i	adjacency and boundary compatibility
B_i	topological compatibility with local CodeHDAG / Metatron projection
S_i	seam consistency with neighboring fragments
R_i	resonance or feature agreement across SourceCubes and SyntheticSourceCubes
P_i	provenance strength and source diversity
G_i	gate admissibility: taint, license, injection, authority, policy
H_i	hysteresis / temporal persistence / prior successful recurrence
F_i	Foundry-relevant feasibility: compile/test/lint likelihood or prior proof
M_i	memory support: StructuralCrystals, NormGenes, MicroTopologyIndex relations

Normalize:

$$w_i^U = \frac{\tilde{w}_i^U}{\sum_j \tilde{w}_j^U} \tag{7.2}$$

If the denominator is zero, the patch is non-condensable.

7.2 Dominance

$$Z_U = \max_i w_i^U \tag{7.3}$$

A local condensation bit activates only if:

$$Z_U \geq \theta_+ \tag{7.4}$$

where $\theta_+ > 0.5$. It deactivates when $Z_U \leq \theta_-$. This hysteresis prevents flip noise.

Gate SUPPORT-001: No support without provenance. Evidence without provenance **MUST NOT** contribute to support mass.

Gate SUPPORT-002: No majority without normalized support. A 51 percent bit **MUST** be computed only from a normalized support vector.

Gate SUPPORT-003: Gate factor zeroing. A hard negative gate **MAY** set $G_i = 0$, making the direction inadmissible regardless of other support factors.

8 Local Condensation Candidate

Type LocalCondensationCandidate.

```
pub struct LocalCondensationCandidate {
    pub id: Digest,
    pub fragment_id: Digest,
    pub selected_direction: CandidateDirectionId,
    pub support_vector: SupportMassVector,
    pub dominance: Q16,
    pub threshold: Q16,
    pub status: LocalCondensationStatus,
    pub triangulation_digest: Digest,
    pub evidence_bundle_id: Digest,
}

pub enum LocalCondensationStatus {
    Dormant,
    CondensedLocal,
    SeamLinked,
    Percolating,
    CoarseGrained,
    Rejected,
    DiagnosticHold,
}
```

Requirement COND-001: Candidate not commit. A LocalCondensationCandidate is a candidate. It is not a commit, patch, trusted memory object, or truth claim.

Requirement COND-002: Diagnostic hold. If dominance fails, support denominator is zero, topology guard fails, or semantic loss is too high, the fragment **MUST** emit a diagnostic hold with a failure reason.

9 Seam Percolation

9.1 Seam graph

Local candidates become useful only when compatible across fragment boundaries.

Type SeamPercolationGraph.

```
pub struct SeamPercolationGraph {
    pub id: Digest,
    pub candidates: Vec<LocalCondensationCandidateId>,
    pub seam_edges: Vec<SeamEdge>,
    pub percolative_paths: Vec<PercolativePath>,
    pub replay_descriptor: ReplayDescriptor,
}

pub struct SeamEdge {
    pub from: LocalCondensationCandidateId,
    pub to: LocalCondensationCandidateId,
    pub compatibility: Q16,
    pub compatibility_factors: SeamCompatibilityFactors,
    pub status: SeamStatus,
}
```

9.2 Compatibility factors

Seam compatibility includes:

- boundary agreement;
- role and layer continuity;
- carrier/fiber continuity;
- absence of contradictions;
- compatible provenance and replay identity;
- support-direction agreement;
- allowed symmetry, duality, or antiphase relations;
- Foundry feasibility across the seam.

Gate SEAM-001: No percolation without seam. A local condensation candidate **MUST NOT** be lifted into a percolative path unless seam compatibility passes declared threshold γ.

Gate SEAM-002: False percolation hold. A geometrically connected path that fails provenance, replay, taint, or validation constraints **MUST** be marked as FalsePercolation and held.

10 Coarse-Graining

10.1 Meso-level aggregation

A compatible percolative path may be coarse-grained into a higher-order candidate. Aggregation must be majority-monotone.

$$\bar{w} = \sum_j \pi_j w^{U_j} \tag{10.1}$$

$$Z_{agg} = \max_i \bar{w}_i \tag{10.2}$$

10.2 Majority monotonicity

Requirement CG-001: Majority monotonicity. Increasing admissible support for a candidate at a lower scale **MUST NOT** reduce its aggregate support at the next scale unless an explicit negative gate fires.

Requirement CG-002: Provenance preservation. Coarse-graining **MUST** preserve provenance of each local contribution. Aggregates without traceable local support are invalid.

Type CoarseGrainedCollapseCandidate.

```
pub struct CoarseGrainedCollapseCandidate {
    pub id: Digest,
    pub source_path: PercolativePathId,
    pub scale: CollapseScale,
    pub aggregate_support: SupportMassVector,
    pub dominance: Q16,
    pub constituent_fragments: Vec<FragmentPatchId>,
    pub candidate_plan_ref: Option<HostTargetCollapsePlanId>,
    pub evidence_bundle_id: Digest,
}
```

Part III

Agentic Corpus Cultivation

11 Synthetic SourceCube Cultivation

11.1 Motivation

External corpora may be sparse, low-quality, incoherent, or unavailable. HYPHAE may therefore request controlled synthetic artifacts from one or more agents. These artifacts become SyntheticSourceCubes and enter the same collapse pipeline as external evidence, but with explicit synthetic provenance and lower initial authority.

> **Agents generate variation. HYPHAE performs selection. Kosmocrates stores inheritance. Foundry tests reality.**

11.2 SyntheticSourceCube

Type SyntheticSourceCube.

```
pub struct SyntheticSourceCube {
    pub id: Digest,
    pub generating_agent: AgentDescriptor,
    pub generation_prompt_digest: Digest,
    pub task_fragment_id: FragmentPatchId,
    pub artifact_digest: Digest,
    pub code_hdag_digest: Option<Digest>,
    pub declared_intent: String,
    pub authority: AuthorityClass,
    pub taint: TaintLabel,
    pub validation_state: SyntheticValidationState,
}
```

11.3 Agent roles

Agent role	Purpose
Variant Generator	Produces multiple local solutions to one fragment.
Critic Agent	Produces objections, edge cases, and negative evidence.
Test Agent	Generates test motifs and validation templates.
Security Agent	Generates threat models and boundary concerns.
Refactor Agent	Proposes structural transformations.
Documentation Agent	Produces neutral descriptions and interface explanations.
Adversarial Agent	Attempts to find contradictions, non-seams, and fragile assumptions.

Requirement SYN-001: Synthetic artifacts are untrusted. Artifacts generated by agents **MUST** be treated as synthetic external evidence. They **MUST NOT** be trusted merely because they were generated inside the system.

Requirement SYN-002: Fragment scope. Agent-generated artifacts **SHOULD** be small, fragment-bound, independently parseable, and independently validatable.

Requirement SYN-003: Diversity over authority. Multiple agents or projection prompts **MAY** be used to increase candidate diversity. Diversity increases search coverage, not authority.

12 Good Fragmentation Protocol

12.1 Fragment quality criteria

A fragment is good if it is:

- bounded;
- independently inspectable;
- independently testable where possible;
- comparable to external or synthetic evidence;
- capable of producing candidate directions;
- connected to seams;
- small enough for local reasoning;
- large enough to preserve relevant semantics.

12.2 Fragmentation objective

> **Maximize useful degrees of freedom while minimizing spurious degrees of freedom after gate contraction.**

12.3 Fragmentation failure modes

Failure	Meaning
OverFragmentation	Fragments are too small and semantic loss dominates.
UnderFragmentation	Fragments are too large and do not locally collapse.
BoundaryLeak	Fragment boundary cuts through a required invariant.
NoSeamModel	Fragments cannot be recomposed.
EvidenceStarvation	A fragment has too little admissible evidence.
AgentNoise	Synthetic artifacts dominate without validation.
GateBlindness	A fragment appears good only because gates are not applied.

Gate FRAG-QUALITY-001: Semantic loss bound. Fragmentation **MUST** record semantic loss. A fragment with excessive semantic loss **MUST** be held or expanded.

Gate FRAG-QUALITY-002: Recomposition requirement. A fragment set **MUST** define seam and recomposition semantics before local candidates may be coarse-grained.

13 Monotone Contractive Filtering

13.1 Filter sequence

A conformant implementation applies a monotone filter sequence:

```
S0 = all candidate directions
S1 = provenance admissible
S2 = topology admissible
S3 = taint/license/security admissible
S4 = support-positive
S5 = local majority condensed
S6 = seam-compatible
S7 = coarse-grain compatible
S8 = Workbench materializable
S9 = Foundry validated
S10 = memory promotable or evidence-only
```

Each step must either preserve or reduce the candidate set, except where a documented refinement expands one candidate into typed sub-candidates with preserved provenance.

Invariant MONO-001: No untracked expansion. No filtering stage may introduce new candidate directions without a traceable derivation from an earlier candidate, evidence item, or operator rule.

Invariant MONO-002: Expansion requires refinement identity. If a candidate is split into sub-candidates, the split **MUST** preserve parent identity and explain why refinement is monotone rather than unconstrained expansion.

13.2 Spurious DoF reduction

Type DoFContractionReport.

```
pub struct DoFContractionReport {
    pub run_id: Digest,
    pub initial_candidate_count: usize,
    pub candidate_counts_by_stage: BTreeMap<FilterStage, usize>,
    pub removed_by_stage: BTreeMap<FilterStage, Vec<CandidateDirectionId>>,
    pub retained_candidates: Vec<CandidateDirectionId>,
    pub contraction_ratio: Q16,
    pub false_local_collapse_rate: Q16,
}
```

Requirement MONO-003: Report contraction. Every LPCM-enabled HYPHAE run **SHOULD** emit a DoFContractionReport.

Part IV

Fixpoint Surfing and Collapse Planning

14 Fixpoint Surfing

14.1 Definition

Fixpoint surfing is a staged collapse procedure in which each validated local or meso-level condensation constrains neighboring fragments, causing the solution space to contract progressively toward stable host-level plans.

```
local collapse -> seam constraint update -> adjacent support shift
-> new local collapse -> percolative path -> coarse-grained candidate
-> collapse plan -> Foundry validation -> memory feedback
```

14.2 Coupling update

A local condensation may increase coupling to compatible neighbors only through bounded and traceable coupling channels.

$$\kappa_{UV}(t^+) = \kappa_{UV}(t)(1 + \rho\beta_U(t)) \tag{14.1}$$

where ρ is bounded by policy.

Gate FIX-001: Bounded coupling. Local collapse feedback **MUST** be bounded, traceable, and reversible. It **MUST NOT** bypass seam, replay, validation, or authority gates.

Gate FIX-002: Oscillation control. If a fragment repeatedly flips between candidate directions, hysteresis **MUST** place it in DiagnosticHold or require additional evidence.

15 Collapse Planning

15.1 HostTargetCollapsePlan

Type HostTargetCollapsePlan.

```
pub struct HostTargetCollapsePlan {
    pub id: Digest,
    pub host_id: Digest,
    pub source_condensates: Vec<CoarseGrainedCollapseCandidateId>,
    pub planned_steps: Vec<CollapsePlanStep>,
    pub seam_graph_digest: Digest,
    pub expected_topology_delta: ExpectedTopologyDelta,
    pub required_foundry_checks: Vec<FoundryCheck>,
    pub risk_model: RiskModel,
    pub evidence_bundle_id: Digest,
}
```

Type CollapsePlanStep.

```
pub struct CollapsePlanStep {
    pub id: Digest,
    pub step_kind: CollapseStepKind,
    pub target_region: TopologyRegionRef,
    pub backing_candidates: Vec<LocalCondensationCandidateId>,
    pub surgery_option: Option<TopologicalSurgeryOptionId>,
    pub preconditions: Vec<Precondition>,
    pub validation_template: ValidationTemplate,
    pub rollback_plan: RollbackPlan,
}
```

15.2 Plan semantics

A HostTargetCollapsePlan is a plan, not a patch. It may be given to the Workbench for materialization only after capability and operator gates pass.

Gate PLAN-001: No materialization without Workbench. Collapse plans **MUST NOT** mutate host code directly. Workbench and Foundry must materialize and validate.

Gate PLAN-002: Foundry binding. Every executable CollapsePlanStep **MUST** declare Foundry checks before materialization.

Part V

Integration

16 Metatron Microtopology Integration

16.1 Role

Metatron v0.4.1 provides local region projection, fingerprints, diagnostics, and surgery options. LPCM uses those projections as triangulated local patches and uses surgery options as candidate directions.

```
TopologyRegionRef
-> MetatronMicrograph
-> MetatronRegionFingerprint
-> MicroTopologyDiagnostic
-> CandidateDirection[]
-> LPCM support mass
-> LocalCondensationCandidate
```

Requirement MET-001: Semantic loss participation. SemanticLossRecord **MUST** influence support mass and gate admissibility. High semantic loss should reduce or zero support for candidate directions that rely on lost semantics.

Requirement MET-002: Diagnostic is evidence. A Metatron diagnostic is evidence. It is not authority. It may contribute to support mass but cannot bypass majority, seam, Foundry, or governance gates.

17 CubeSwarm and Mandorla Integration

17.1 Position

CubeSwarm and CubeMandorla provide multi-source structural support. LPCM specifies how this support becomes local dominance and percolative collapse.

```
SourceCubes + SyntheticSourceCubes
-> local support factors R, P, M, F
-> support vectors per fragment
-> local majority bits
-> seam graph
-> coarse-grained collapse plan
```

Requirement CUBE-001: Support is local before global. CubeSwarm support **MUST** be localized to fragments before it is used for percolative collapse. Global-looking support without local patch grounding **MUST** be treated as weak evidence.

18 Kosmocrates Memory Integration

18.1 Memory targets

LPCM-enabled runs may produce:

- EvidenceOnlyCrystals;
- StructuralCrystalCandidates;
- NormGene fitness updates;
- MicroTopologyIndex entries;
- CollapsePlan traces;
- negative evidence records;
- DoFContractionReports;
- PercolationPath records.

Requirement MEM-001: Evidence first. A local or percolative condensation result **MUST** enter memory first as evidence or candidate memory. It **MUST NOT** become trusted norm memory by local majority alone.

Requirement MEM-002: Negative evidence retention. Failed local collapses, seam breaks, oscillatory flips, and false percolations **SHOULD** be retained as negative evidence for future runs.

Part VI

API, CLI, Metrics

19 API Extensions

19.1 Endpoints

```
POST /api/hyphae/lpcm/runs
GET  /api/hyphae/lpcm/runs/{id}
GET  /api/hyphae/lpcm/runs/{id}/fragments
GET  /api/hyphae/lpcm/runs/{id}/support
GET  /api/hyphae/lpcm/runs/{id}/seams
GET  /api/hyphae/lpcm/runs/{id}/paths
GET  /api/hyphae/lpcm/runs/{id}/collapse-plan
GET  /api/hyphae/lpcm/runs/{id}/contraction-report
POST /api/hyphae/lpcm/runs/{id}/materialize
```

19.2 Run request

```
pub struct LpcmRunRequest {
    pub host_id: Digest,
    pub goal_id: Digest,
    pub fragment_policy: FragmentPolicy,
    pub candidate_policy: CandidatePolicy,
    pub support_policy: SupportPolicy,
    pub seam_policy: SeamPolicy,
    pub coarse_grain_policy: CoarseGrainPolicy,
    pub synthetic_cultivation_policy: Option<SyntheticCultivationPolicy>,
    pub dry_run: bool,
}
```

20 CLI Extensions

```
kosmocrates hyphae lpcm run --host . --goal "add robust integration tests"
kosmocrates hyphae lpcm fragments <run-id>
kosmocrates hyphae lpcm support <run-id>
kosmocrates hyphae lpcm seams <run-id>
kosmocrates hyphae lpcm collapse-plan <run-id>
kosmocrates hyphae lpcm contraction-report <run-id>
kosmocrates hyphae lpcm materialize <run-id> --step <step-id>
```

21 Metrics

21.1 Required metrics

Metric	Meaning
FragmentCount	Number of deterministic local patches.
TriangulablePatchRate	Fraction of patches passing topology construction.
CandidateDirectionCount	Number of candidate directions per patch.
SupportPositiveRate	Fraction of patches with positive admissible support.
LocalCondensationRate	Fraction of patches crossing local majority.
MeanDominanceMargin	Mean $Z_U - \theta$ among activated patches.
SeamConsistencyRate	Fraction of local candidate pairs passing seam threshold.
PercolationPathStrength	Weighted support of accepted percolative paths.
CoarseGrainStability	Replay-stable persistence of aggregate condensates.
FalseLocalCollapseRate	Local activations rejected by higher gates.
DoFContractionRatio	Candidate reduction from initial space to retained plan.
SyntheticContributionRate	Share of support mass contributed by synthetic artifacts.
FoundrySurvivalRate	Share of materialized collapse steps passing Foundry.
ReplayIdentity	Bit-identical reconstruction status.

Part VII

Implementation Plan

22 Phased Implementation

22.1 Phase L0: Passive LPCM report

Implement FragmentField, FragmentPatch, CandidateDirection, SupportMassVector, LocalCondensationCandidate, and DoFContractionReport. Do not materialize anything.

22.2 Phase L1: Metatron-backed local patches

Use MetatronMicrograph output as local triangulation. Convert MicroTopologyDiagnostics into candidate directions.

22.3 Phase L2: Support mass and 51 percent gate

Compute support factors, normalized support vectors, hysteretic activation, local candidates, and dormant/hold statuses.

22.4 Phase L3: Seam graph and percolation

Build seam compatibility graph, detect percolative paths, and emit false-percolation holds.

22.5 Phase L4: Coarse-graining and collapse plans

Aggregate percolative paths into HostTargetCollapsePlans with Foundry validation templates.

22.6 Phase L5: Synthetic corpus cultivation

Allow controlled agents to generate fragment-bound SyntheticSourceCubes. Treat them as low-authority evidence.

22.7 Phase L6: Workbench materialization

Allow operator-approved collapse steps to enter Workbench and Foundry. Feed results back to memory.

23 Acceptance Tests

23.1 Unit tests

```
AT-LPCM-001 provenance missing -> support contribution rejected
AT-LPCM-002 non-triangulable patch -> DiagnosticHold
AT-LPCM-003 zero denominator -> ZeroSupport hold
AT-LPCM-004 support vector normalized to sum 1
AT-LPCM-005 no local condensation below theta
AT-LPCM-006 local condensation above theta_plus
AT-LPCM-007 hysteresis prevents flip at theta_minus < Z < theta_plus
AT-LPCM-008 local majority does not create patch authority
AT-LPCM-009 seam edge rejected below gamma
AT-LPCM-010 percolative path requires all nodes locally condensed
AT-LPCM-011 coarse-grain preserves provenance
AT-LPCM-012 majority monotonicity holds unless negative gate fires
AT-LPCM-013 synthetic artifact has low authority and taint
AT-LPCM-014 raw synthetic output cannot become trusted memory
AT-LPCM-015 Foundry failure reduces future support through negative evidence
```

23.2 Integration tests

```
IT-LPCM-001 host with missing service boundary:
  fragments route/handler/db region,
  Metatron detects direct Handler->Db edge,
  LPCM condenses InsertMediatorRole,
  seam percolates into boundary collapse plan.

IT-LPCM-002 synthetic variants:
  three agents generate service-boundary variants,
  two variants align with host and Foundry template,
  one is rejected by taint/license/semantic-loss gate,
  support condenses only over admissible variants.

IT-LPCM-003 no good external corpus:
  external sources provide weak or conflicting support,
  no local majority forms,
  diagnostic hold and negative evidence are emitted.

IT-LPCM-004 second similar host:
  prior StructuralCrystal/NormGene evidence increases memory factor M_i,
  fewer external acquisitions are needed,
  collapse plan forms faster.
```

24 Definition of Done

HYPHAE v0.4.2 LPCM Extension is complete when:

1. FragmentField partitioning is deterministic and replayable.
2. Local patches can be backed by MetatronMicrographs or CodeHDAG regions.
3. CandidateDirections are typed, evidenced, and non-authoritative.
4. Support mass vectors are normalized and gate-aware.
5. The 51 percent gate emits candidates only, never truth.
6. Hysteresis prevents oscillatory flip noise.
7. Seam graph and percolative paths are computed with explicit compatibility thresholds.
8. Coarse-graining preserves provenance and obeys majority monotonicity.
9. DoFContractionReport is emitted for each run.
10. SyntheticSourceCubes are supported but treated as untrusted evidence.
11. CollapsePlans are materialized only through Workbench and Foundry.
12. Negative evidence is retained.
13. All AT-LPCM and IT-LPCM tests pass.

25 Engineering Notes

25.1 Do not overbuild first

The first implementation should not attempt global optimization. Implement passive reporting first:

```
Host region -> fragments -> candidate directions -> support mass
-> local condensation bits -> seam graph -> report
```

Only after the report is deterministic and useful should materialization be connected.

25.2 Strict boundaries

- Do not allow LPCM to write host files.
- Do not allow agent-generated artifacts to become trusted memory directly.
- Do not treat 51 percent dominance as correctness.
- Do not hide semantic loss.
- Do not aggregate local candidates without seams.
- Do not coarse-grain without provenance.

25.3 Minimal MVP target

> Given a host repository with a known local architecture defect, produce a deterministic LPCM collapse report that identifies fragments, candidate directions, support mass, local condensation bits, seam compatibility, and a planning-only CollapsePlanStep. No code changes are required for the first MVP.

Closing Thesis

HYPHAE v0.4.1 gave Kosmocrates local topological diagnostics and surgery planning. This extension supplies the collapse mechanics that make good fragmentation useful: the system first expands a host problem into many inspectable degrees of freedom, then monotonically reduces spurious freedom through evidence, topology, support, seam, validation, and replay until a bounded structural condensation occurs.

> **Fragmentation expands the admissible phase space. Monotone filtering contracts spurious degrees of freedom. Local dominance emits condensation candidates. Seam percolation lifts them into collapse plans. Foundry and Kosmocrates decide what becomes real.**